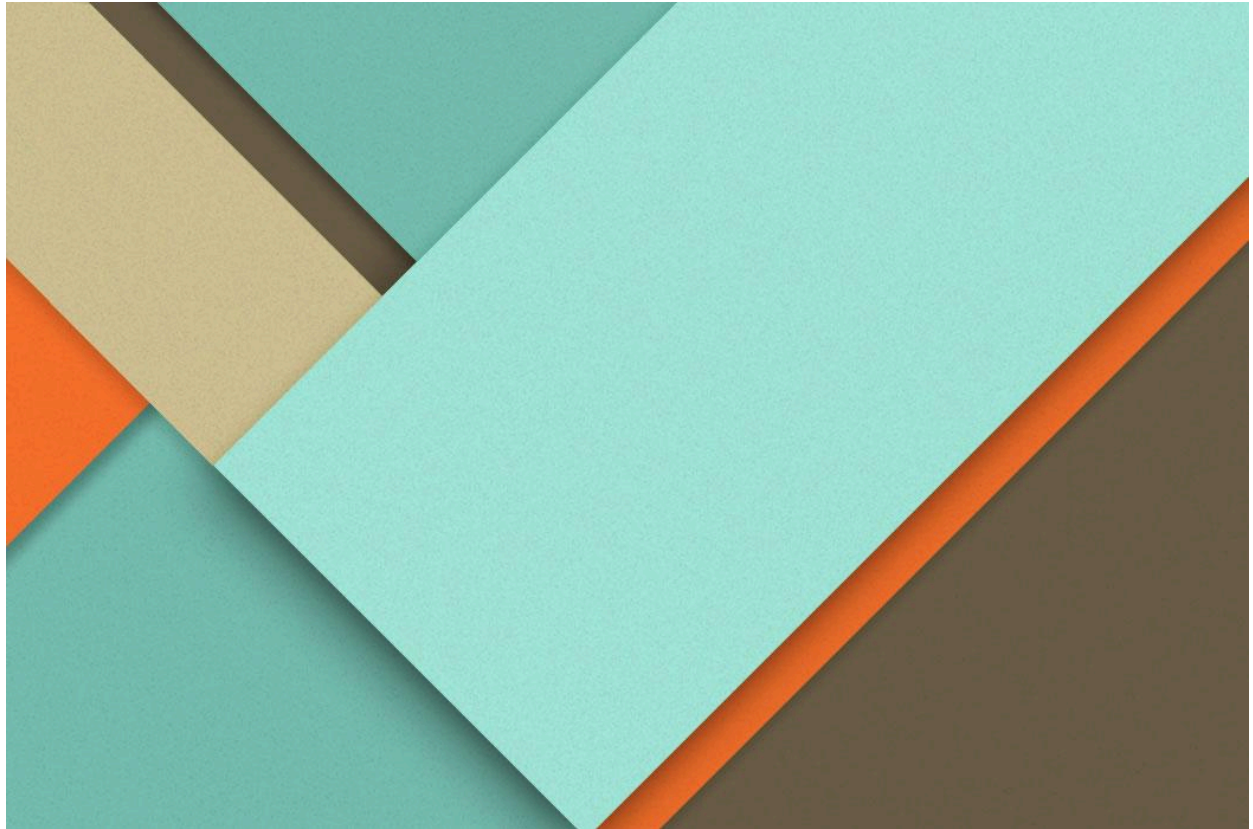




SonarQube Proyecto GYM

13/05/2026

Unai Rafales

Marcos Ballesteros

Grupo A

Índice

Índice	1
Introducción	2
Semana 1 - CRUD Usuarios	3
Semana 2 - CRUD Maquinaria, Salas, Actividades, Actividades Programadas y JSON para Web	4

Introducción

En el desarrollo de un proyecto de software para la gestión de un gimnasio implementado en Java, la calidad del código, la mantenibilidad y la detección temprana de errores son aspectos fundamentales para garantizar un sistema robusto y escalable. En este contexto, se ha optado por la utilización de SonarQube como herramienta de análisis estático de código, debido a su capacidad para evaluar de forma automática la calidad del software en múltiples dimensiones.

SonarQube permite identificar problemas comunes como errores potenciales, vulnerabilidades de seguridad, duplicación de código y malas prácticas de programación, proporcionando además métricas claras que ayudan al equipo de desarrollo a mejorar continuamente la estructura del proyecto. Su integración en un entorno Java resulta especialmente útil, ya que facilita el seguimiento de estándares de codificación y asegura que el sistema cumpla con buenas prácticas desde las primeras fases del desarrollo.

El uso de SonarQube en este proyecto de gestión de gimnasio no solo contribuye a reducir la deuda técnica, sino que también mejora la legibilidad y escalabilidad del código, aspectos esenciales en una aplicación que puede evolucionar con nuevas funcionalidades como gestión de socios, control de pagos o planificación de entrenamientos. En definitiva, esta herramienta se convierte en un apoyo clave para garantizar un producto final más fiable, eficiente y profesional.

Semana 1 - CRUD Usuarios

Durante la primera semana del proyecto del gimnasio en Java se ha desarrollado la funcionalidad CRUD (crear, leer, actualizar y eliminar) para la entidad de usuarios.

Se implementaron las operaciones necesarias para registrar nuevos usuarios, consultar la lista y los detalles de cada uno, modificar su información y eliminarlos cuando sea necesario. Esta fase permitió establecer la base del sistema de gestión de usuarios, asegurando una estructura funcional y preparada para futuras ampliaciones del proyecto.

```
daw@DAWI:~/Proyecto_GYM-Unai-Marcos$ /home/daw/sonar-scanner-cli-8.0.1.6346-linux-x64/sonar-scanner-8.0.1.6346-linux-x64/bin/sonar-scanner
11:40:00.238 INFO Scanner configuration file: /home/daw/sonar-scanner-cli-8.0.1.6346-linux-x64/sonar-scanner-8.0.1.6346-linux-x64/conf/sonar-scanner.properties
11:40:00.243 INFO Project root configuration file: /home/daw/.local/share/Trash/files/Proyecto_GYM-Unai-Marcos.31/sonar-project.properties
11:40:00.272 INFO SonarScanner CLI 8.0.1.6346
11:40:00.278 INFO Linux 6.8.0-111-generic amd64
11:40:04.096 INFO Communicating with SonarQube Community Build 26.5.0.122743
11:40:04.098 INFO JRE provisioning: os[linux], arch[x86_64]
11:40:04.527 INFO Starting SonarScanner Engine...
11:40:04.530 INFO Java 21.0.9 Eclipse Adoptium (64-bit)
11:40:08.398 INFO Load global settings
11:40:08.495 INFO Load global settings (done) | time=96ms
11:40:08.498 INFO Server id: 1478411E-AZ5Eai-9LqUHff5a50sz
11:40:08.520 INFO Loading required plugins
11:40:08.520 INFO Load plugins index
11:40:08.534 INFO Load plugins index (done) | time=15ms
11:40:08.536 INFO Load/download plugins
11:40:08.567 INFO Load/download plugins (done) | time=32ms
11:40:08.882 INFO Process project properties
11:40:08.899 INFO Process project properties (done) | time=17ms
11:40:08.916 INFO Project key: Proyecto_GYM-Unai-Marcos
11:40:08.916 INFO Base dir: /home/daw/.local/share/Trash/files/Proyecto_GYM-Unai-Marcos.31
11:40:08.916 INFO Working dir: /home/daw/.local/share/Trash/files/Proyecto_GYM-Unai-Marcos.31/.scannerwork
11:40:08.926 INFO Load project settings for component key: 'Proyecto_GYM-Unai-Marcos'
11:40:08.944 INFO Load project settings for component key: 'Proyecto_GYM-Unai-Marcos' (done) | time=16ms
11:40:08.975 INFO Load quality profiles
11:40:09.032 INFO Load quality profiles (done) | time=56ms
11:40:09.086 WARN SCM provider autodetection failed. Please use "sonar.scm.provider" to define SCM of your project, or disable the SCM Sensor in the project settings
11:40:09.102 INFO Load active rules
11:40:09.564 INFO Load active rules (done) | time=463ms
11:40:09.576 INFO Load analysis cache
```

Resultado:

Proyecto-GYM-Unai-Marcos
Public

Last analysis: 19 seconds ago • 1.7k Lines of Code • Java

2	14	239	0.0%	0.0%	10.4%
Security	Reliability	Maintainability	Hotspots Reviewed	Coverage	Duplications

1 of 1 shown

Pero todavía no estaba completo las operaciones CRUD de usuarios. Así que el lunes lo completamos y este sería el resultado final:



Semana 2 - CRUD Maquinaria, Salas, Actividades, Actividades Programadas y JSON para Web

Durante la segunda semana del proyecto se han implementado las operaciones CRUD de las entidades principales del sistema de gestión del gimnasio en Java: **Maquinaria, Salas, Actividades y Actividades Programadas**. Estas funcionalidades permiten crear, consultar, actualizar y eliminar información clave para el correcto funcionamiento del centro, facilitando la organización de recursos, espacios y planificación de actividades.

Además, se ha incorporado el **inicio de sesión**, que permite el acceso seguro al sistema según el tipo de usuario. A partir de esta autenticación, se han desarrollado los distintos **homes o paneles principales** para cada rol: **administrador, entrenador, recepcionista y socio**, adaptando la interfaz y las funcionalidades disponibles según los permisos de cada uno.

Por último, se ha comenzado la integración de **JSON para la comunicación con la web**, lo que permitirá el intercambio de datos entre el backend en Java y posibles interfaces externas de forma estructurada y eficiente.

```
11:57:58.434 INFO Analysis total time: 11.616 s
11:57:58.436 INFO SonarScanner Engine completed successfully
11:57:58.528 INFO EXECUTION SUCCESS
11:57:58.529 INFO Total time: 19.454s
daw@DAW1:~/Proyecto_GYM-Unai-Marcos$
```

Resultado:



Vimos que tenía varios errores y lo corregimos. Los errores eran que El warning indica que si ocurre una excepción entre prepareStatement() y ps.close(), el PreparedStatement (y la Connection) nunca se cierran, causando fugas de memoria y conexiones colgadas en la base de datos.

La solución que ofrecimos fue en vez hacer en los DAO solo un throw de SQLException que se hiciera un try

```
public static void insertarActividad(Actividad act) throws SQLException {
    String sql = "INSERT INTO actividad (nombre, descripcion, nivel) VALUES (?, ?, ?)";
    try (Connection conn = Conexion.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, act.getNombre());
        ps.setString(2, act.getDescripcion());
        ps.setString(3, act.getNivel());
        ps.executeUpdate();
    }
}

public static void modificarActividad(Actividad act) throws SQLException {
    String sql = "UPDATE actividad SET nombre = ?, descripcion = ?, nivel = ? WHERE id_actividad = ?";
    try (Connection conn = Conexion.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, act.getNombre());
        ps.setString(2, act.getDescripcion());
        ps.setString(3, act.getNivel());
        ps.setInt(4, act.getId());
        ps.executeUpdate();
    }
}

public static void eliminarActividad(int id_act) throws SQLException {
    try (Connection conn = Conexion.getConnection();
        PreparedStatement ps1 = conn.prepareStatement("DELETE FROM actividad_programada WHERE id_actividad = ?");
        PreparedStatement ps2 = conn.prepareStatement("DELETE FROM actividad_sala WHERE id_actividad = ?");
        PreparedStatement ps3 = conn.prepareStatement("DELETE FROM actividad_entrenador WHERE id_actividad = ?");
        PreparedStatement ps4 = conn.prepareStatement("DELETE FROM actividad WHERE id_actividad = ?")) {

        ps1.setInt(1, id_act);
        ps1.executeUpdate();
    }
}
```

Resultado final:



Semana 3 - CRUD Pagos, Nóminas, Reservas y vista completa de socios y administradores

Durante esta semana también se implementaron las operaciones CRUD de pagos, reservas y nóminas, permitiendo registrar, consultar, modificar y eliminar información desde la aplicación. En el módulo de pagos se desarrolló la gestión de cuotas y pagos realizados por los socios del gimnasio, permitiendo llevar un control económico básico dentro del sistema. En el apartado de reservas se añadió la posibilidad de gestionar las reservas de actividades y salas, controlando la disponibilidad y permitiendo a los usuarios apuntarse o cancelar actividades programadas. Respecto a las nóminas, se implementó una gestión sencilla de los pagos realizados a los trabajadores del gimnasio, almacenando información relacionada con salarios y fechas de pago.

Además, se completó la vista del socio y del administrador dentro de la aplicación. La vista del socio fue diseñada para permitir a los usuarios consultar sus actividades, reservas, pagos y datos personales de forma sencilla e intuitiva. Por otro lado, la vista del administrador incluye acceso a todas las funcionalidades de gestión del sistema, como administración de usuarios, control de actividades, gestión de maquinaria, reservas y pagos. El objetivo de completar ambas vistas fue mejorar la organización de la aplicación y ofrecer interfaces adaptadas según el tipo de usuario que accede al sistema.

Resultado:



Había dos errores de seguridad donde se recomienda crear un .env para que no se pudiera ver la contraseña ni otros datos de la base de datos a la que nos conectamos.

Resultado final y corregido:

